

백엔드에서 쌓은 판단을 AI Native 개발 방식으로 구조화합니다

Java와 Kotlin으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 왔습니다. 그 과정에서 쌓은 판단을 에이전트·스킬·평가로 구조화해 실제 개발 흐름에서 재사용합니다.

서민재 · Rex Seo

AI-
Native
Backend
Engineer ckdekrn88@gmail.com

GitHub Velog

LinkedIn Rallit

Resume

9 · 6 · 8

내부 이벤트 구조

이벤트 9종이 멈춘 위치를 6개 상태로 남기고, 업무별 수동 복구 전략 8개를 연결했습니다.

2,700→20

피드 랭킹 단계

9개 소스의 후보 상한을 2,700건으로 두고 FPR 500, SPR 100, 최종 20건으로 좁혔습니다.

29 / 43

test-support 구성

29개 helper에 fixture·mock·Web·JPA 준비를 모으고, 43개 테스트로 동작을 확인했습니다.

15 + 32

AI 개발 자산

agent 15개와 skill 32개를 설치·검증 가능한 구조로 묶고 Codex 탐색 32/32를 확인했습니다.

01

POINT PLATFORM · EVENT RELIABILITY

발행 실패와 처리 실패, 같은 방식으로 복구할 수 없었습니다.

포인트 차감이나 회수가 늦게 실패하면 정산 확인과 고객 응대도 함께 길어집니다. 실패 지점을 상태로 기록하고 업무별 복구 경로를 설계해, 운영자가 로그와 원장을 매번 대조하는 일을 줄였습니다.

담당

이벤트 상태·수동 복구·정체
알림 설계 및 구현

구조

내부 이벤트 9종, 상태 6개,
타입별 복구 전략 8개

저장

PostgreSQL
internal_event_record ·
JSONB payload

관측

IN_PROGRESS 15분 초과 건을
타입별로 집계해 Slack 전송

상황

‘실패’ 하나로 묶자, 원인을 찾는 일부터 다시 시작해야 했습니다.

메시지를 보내지 못한 경우와 소비자가 처리 중 멈춘 경우는 확인할 데이터도, 재시도할 위치도 다릅니다. 상태가 같으면 운영자가 로그와 원장을 일일이 맞춰 봐야 했습니다.

판단

멈춘 지점을 기록하고, 복구 로직은 업무별로 나눴습니다.

생성부터 복구 불가 실패까지 6개 상태를 두었습니다. 포인트 차감·회수·만료 알림처럼 조건이 다른 8개 작업은 저장된 payload로 각 업무 서비스를 다시 호출합니다.

멈춘 위치에서 다시 시작하는 복구 흐름

생성부터 처리, 수동 복구와 정체 감지까지

— 호출 — 비동기 — 복구

01 정상 처리

업무 요청과 처리 상태를 기록한 뒤 Kafka로 전달합니다.

업무 요청

API · Admin · Batch

차감 · 회수 · 사용 · 만료

기록

상태 기록

internal_event_record
InternalEventCreator
CREATED · JSONB payload

전달

KAFKA

내부 이벤트 topic 9개

발행 실패는 SEND_FAILED로 분리

처리

업무 처리

Listener 9개 · EventProcessor

IN_PROGRESS → 완료 · 실패

02 수동 복구

Kafka에 다시 넣지 않고 실패한 업무만 실행합니다.

복구 대상 조회

type · status로 검색

저장된 payload 복원

선택

복구 전략

Retry Strategy 8개

이벤트 타입별 업무 분기

재실행

DOMAIN SERVICE

업무 서비스 직접 호출

차감 · 회수 · 사용 · 알림

03 정체 감지

15분 넘게 처리 중인 이벤트를 Slack으로 알립니다.

QUERYDSL

IN_PROGRESS 15분 초과

추적 대상 이벤트 8종

집계

SLACK

이벤트 타입별 요약

운영 채널로 건수 전송

달라진 운영

전송 실패와 처리 실패를 서로 다른 상태로 기록합니다. 처리 중 15분 넘게 멈춘 이벤트는 Slack으로 알리고, 저장된 payload로 실패한 업무만 다시 실행합니다.

세부 구현과 예외 처리

- 내부 이벤트 9종을 생성, 발행 실패, 처리 중, 완료, 처리 실패, 복구 불가 실패의 6개 상태로 기록했습니다.
- 수동 복구 API가 타입과 상태로 레코드를 찾고 저장된 payload로 8개 전략 중 하나를 실행하도록 구성했습니다.
- Kafka에 다시 넣지 않고 타입별 업무 서비스를 직접 호출해 복구 범위를 실패한 업무로 제한했습니다.
- IN_PROGRESS 상태로 15분 이상 멈춘 추적 대상 8종만 조회해 타입별 건수를 Slack에 전송했습니다.
- 상태 전환 타이밍 오류, 잘못된 상태 검사, 이벤트 발송 실패 로깅 누락을 수정했습니다.
- 배치 재시도와 프록시 기반 재시도 호출 문제를 수정하고 포인트 선택 복구 경로를 보강했습니다.

대량 배치는 처리량만큼 재시작 단위가 중요했습니다.

원장 소멸, 만료 안내, 아카이브는 같은 데이터를 보지만 끝내야 할 일은 다릅니다. 한 번에 많이 처리하기보다 중간에 멈춰도 진행 위치를 확인할 수 있도록 조회와 저장 단위를 나눴습니다.

담당	처리 단위	보관 대상	동시성
원장 소멸 페이지 전환·아카이브 배치 구현·만료 알림 이벤트 개선	전체 적재 대신 페이지 단위 반복 처리	point_core 6개·point_service 4개 archive 테이블	분산락, 예산 경합, 유료·무료 원장 혼합 사용

상황

세 Job이 같은 데이터를 만진다고 같은 방식으로 돌릴 수는 없었습니다.

소멸은 잔액과 원장을 바꾸고, 만료 안내는 알림을 쌓아 발송하며, 아카이브는 참조 순서대로 테이블을 옮깁니다. 전체 적재 방식은 데이터가 늘수록 실패 범위도 함께 커졌습니다.

판단

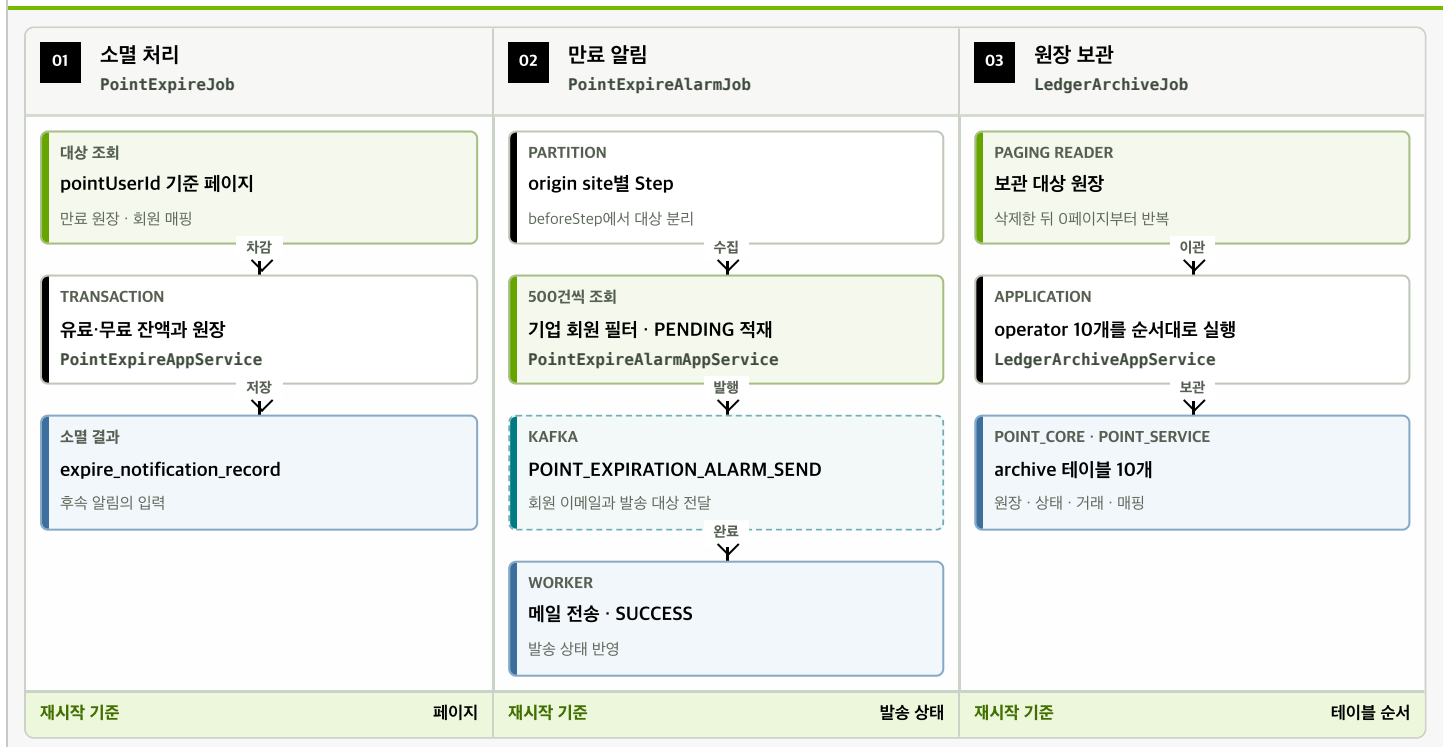
원장은 페이지로 읽고, 후속 작업은 순서를 테스트로 고정했습니다.

소멸은 일정한 페이지 단위로 반복합니다. 만료 대상 수집부터 Kafka 발행까지의 순서와 10개 archive 테이블의 이관 순서는 각각 테스트로 고정했습니다.

세 배치, 세 가지 재시작 기준

같은 원장을 다뤄도 다시 시작할 기준은 서로 다릅니다.

— 순차 처리 - - - 후속 이벤트



실패 후 재시작 기준

소멸은 페이지, 만료 안내는 발송 상태, 아카이브는 테이블 순서를 기준으로 다시 시작합니다. 세 Job마다 확인할 지점이 다릅니다.

- 만료 대상 수집, 날짜 계산, 기업 회원 필터, 만료 알림 레코드 일괄 저장 순서를 검증했습니다.
- point_core 6개와 point_service 4개 테이블의 아카이브 흐름과 테스트를 추가했습니다.
- 동일·상이한 lock key의 동시 요청과 같은 예산 ID의 동시 차감 시나리오를 검증했습니다.
- 유료·무료 원장 혼합 사용 케이스를 별도로 검증했습니다.
- 회원-워크스페이스 매핑, master/member 분리, 회원 탈퇴 이벤트, globalUserRef 중복 처리를 보강했습니다.

03

FEED & SEARCH · SERVING / RETRIEVAL

캐시는 얼마나 빨리 읽느냐보다 언제 무효화하느냐가 더 중요했습니다.

정상 스크롤에서는 후보를 재사용하고, 좌표·필터·로그인 상태가 바뀌면 Snapshot·FPR·SPR 후보 캐시만 무효화했습니다.

후보 조회	랭킹	캐시	분기
OpenSearch 9개 소스 병렬 실행, 설정상 최대 2,700건	FPR 500 → SPR/Re-Rank 100 → 페이지 20건	Snapshot · FPR · SPR · User Vector, TTL 30분	요청 지문 변경, guest → member, Redis 장애

상황

조건이 바뀐 뒤에도 기존 후보를 쓰면 빠르지만 틀린 결과가 됩니다.

좌표나 필터가 달라지면 이전 후보는 현재 요청과 맞지 않습니다. 반대로 페이지마다 새로 계산하면 같은 검색과 랭킹을 되풀이하게 됩니다.

판단

스크롤, 후보 소진, 조건 변경, 장애를 각각 다르게 처리했습니다.

정상 스크롤은 page queue만 읽고 후보가 부족할 때만 refill합니다. 요청 지문이 달라지면 후보 풀을 비우며, Redis를 사용할 수 없으면 캐시 기록을 건너뛰고 OpenSearch 후보를 메모리에서 랭킹해 응답합니다.

피드 캐시를 재사용하고, 무효화하고, 우회하는 기준

Snapshot의 상태와 요청 지문에 따라 네 경로로 나뉩니다.

— 정상 흐름 - - - 외부 조회 — 무효화 · 우회

01 요청 판별

같은 세션에서도 좌표와 필터가 달라지면 다른 요청입니다.

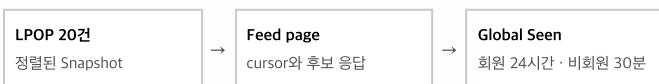


Snapshot과 요청 조건은 지금도 유효한가?

hit · empty · changed · unavailable

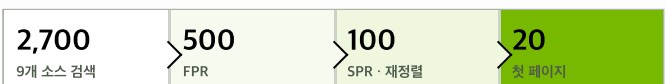
H 캐시 적중

검색과 랭킹을 건너뛸니다.

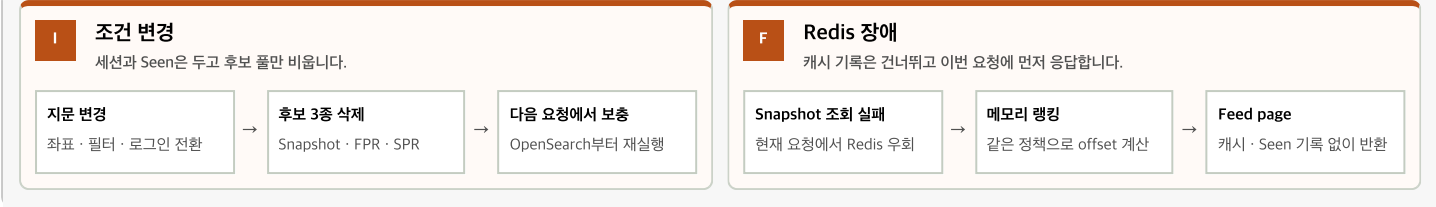


R 후보 보충

Snapshot이 비었을 때만 랭킹을 다시 계산합니다.



최종 100건을 R PUSH하고 첫 20건은 바로 응답합니다.



캐시 판단 기준

정상 스크롤에서는 남은 Snapshot을 재사용하고, 조건이 바뀌면 후보 풀만 무효화합니다. Redis를 쓸 수 없을 때도 같은 랭킹을 메모리에서 계산해 응답합니다.

랭킹·캐시·검색 정책 상세	
- Snapshot, FPR pool, SPR pool, user vector, global seen을 서로 다른 Redis key와 TTL로 관리했습니다. 9개 OpenSearch 소스를 virtual thread에서 병렬 호출하고, 한 소스가 실패하거나 timeout이 나면 해당 결과만 제외해 나머지 후보를 유지했습니다. - Snapshot에 남은 후보가 있으면 20건만 꺼내고, 소진됐을 때만 100건짜리 랭킹 window를 다시 채웠습니다. - seen은 노출한 페이지마다 기록하고 FPR에서 감점 신호로 사용했습니다.	- 세션과 후보 캐시는 30분, 회원 global seen은 24시간 동안 유지했습니다. - 위치 기준 거리 가중치와 10km 반경, 좌표가 없는 광고의 보존·제외 정책을 반영했습니다. - 메인 4축 필터를 색인과 질의 조립에 반영하고 회원·비회원에서 같은 의미로 적용했습니다. - 유료 후보는 별도 영역으로 떼지 않고 FPR 점수 가산에 포함했습니다. - 재게시된 같은 광고는 FPR에서 contentHash로 중복을 제거했습니다. - 검색 인덱스 전환, read/write 계정 경계, strict mapping 이슈를 함께 다뤘습니다.

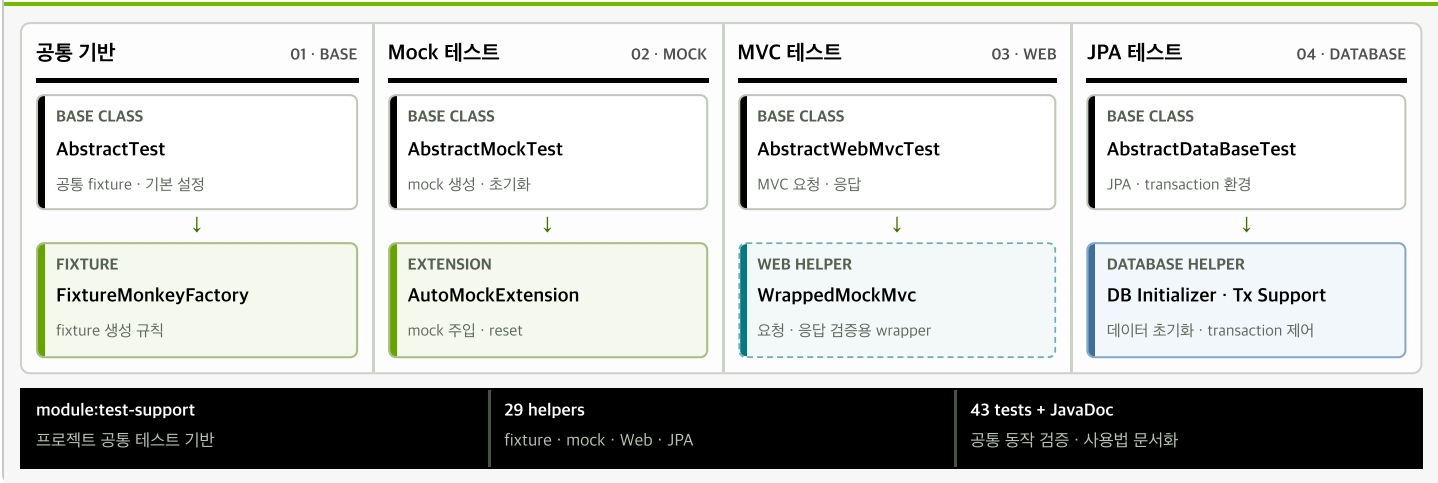
테스트마다 반복하던 준비 코드를 test-support로 모았습니다.

fixture와 mock 준비 방식이 프로젝트마다 달라 리뷰할 때마다 테스트 설정부터 다시 읽어야 했습니다. Web, JPA, mock 등 목적별 진입점을 만들고 반복되는 준비 코드는 helper로 옮겼습니다.

주요 커밋	모듈 구성	테스트	공개 활동
test-support를 확장한 19개 변경	fixture·mock·Web·JPA 등 29개 helper	공통 진입점과 helper를 다룬 43개 테스트	YouthCon 2025 Spring Test 발표

테스트 목적에 맞춰 공통 준비를 나눈 구조

base class와 helper를 네 가지 테스트 방식에 맞춰 묶었습니다.



테스트 기반

19개 주요 변경에 걸쳐 테스트 준비 방식을 29개 helper로 통일했습니다.
fixture·mock·WebMvc·JPA·transaction의 공통 동작은 43개 테스트로 검증합니다.

제공 기능과 CI 처리

- 기본 테스트, mock 테스트, WebMvc 테스트, JPA 테스트의 시작점을 분리했습니다.
 - 공통 fixture 설정과 자동 mock 주입·초기화로 반복 setup을 helper로 옮겼습니다.
- MockMvc 요청 빌더, query string null 처리, 응답 검증 래퍼를 제공했습니다.
 - JPA test initializer, Spring utils, transaction helper로 데이터 테스트 준비를 정리했습니다.
 - 느린 테스트 제외, active profile 기본값, unsafe warning 제거, Javadoc을 반영했습니다.

05

AI ENGINEERING · AGENT LAB V0.1.0.0

AI 에이전트의 역할·지식·절차와 평가 기준을 함께 관리하도록 구조를 설계했습니다.

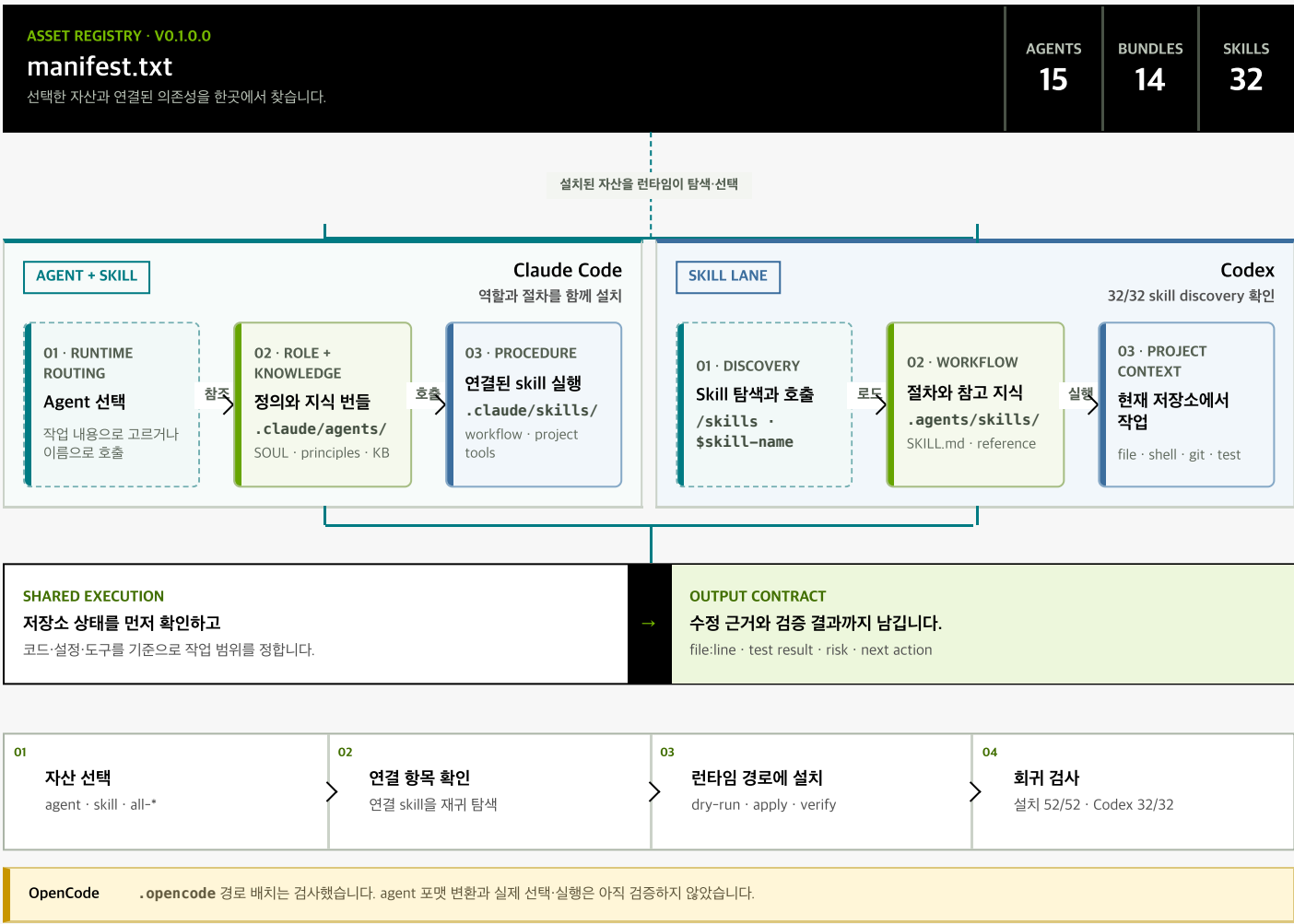
코드 탐색, 설계 검토, 테스트 작성처럼 반복되는 작업을 agent와 skill로 나눴습니다. manifest로 설치 관계를 관리하고, 구조 fixture와 행동 계약, 설치 회귀와 Codex 탐색을 각각 검증했습니다.

AGENT 구성	연결 구조	정적 계약 검증	설치·탐색
15 agents · 32 skills	24 agent→skill · 11 deps	구조 7/7 · 행동 10/10	설치 52/52 · 탐색 32/32

Agent가 역할을 정하고, skill이 절차를 제공하며, KB가 판단 근거를 보탭니다.

Agent Lab이 직접 작업을 실행하지는 않습니다. 자산과 의존성을 묶어 각 런타임에 설치합니다.

— 실행 순서 - - - 런타임 경계



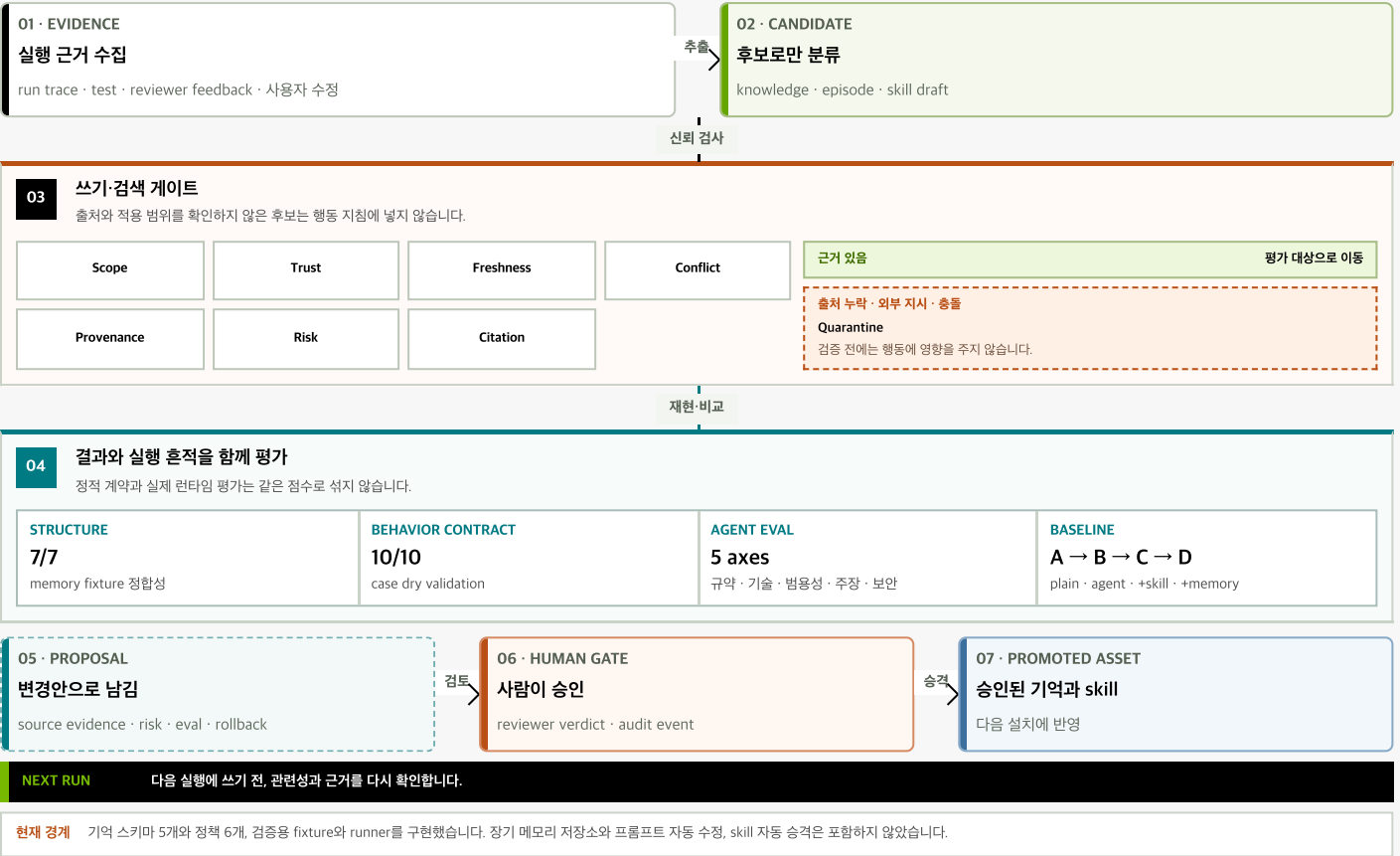
실제 적용

Claude Code에는 agent와 skill을 함께 설치하고, Codex에는 32개 skill을 공식 경로로 배포합니다. 개선안은 자동 반영하지 않습니다. 제안과 평가 기록을 남기고 사람이 승인한 변경만 다음 프로젝트에 적용하도록 설계했습니다.

실행 경험은 곧바로 규칙이 되지 않습니다.

후보의 신뢰·재현성·위험을 확인하고, 사람이 승인한 변경만 다음 실행에 쓰도록 설계했습니다.

승격 후보 격리



도구 범위와 안전 경계

- manifest에 15개 agent, 14개 지식 번들, 32개 skill과 연결 의존성을 등록했습니다.
- 설치 회귀 52개를 통과했고, Codex CLI에서 32개 skill을 모두 탐색했습니다.
- 구조 fixture 7개와 행동 케이스 10개의 입력·임계값·안전 계약을 정적 검증했습니다.
- 행동 평가에서는 결과와 실행 지표를 따로 받아 결과·과정·안전·비용을 비교합니다. 실제 에이전트를 실행하려면 런타임별 adapter가 더 필요합니다.
- 메모리 5개 schema와 6개 policy로 기억을 쓰고 읽는 조건, 격리·승격·교체, 자기 수정의 경계를 먼저 정했습니다.
- NL-SQL MCP에 multi-dialect와 읽기 전용 경계를 두고 위험한 쿼리 실행을 막았습니다.
- code cartographer와 diagram extract skill로 코드 이해 결과를 다이어그램으로 공유하게 했습니다.

테스트, 인증, 검색 경험이 지금의 설계 기준이 되었습니다.

실제 구현을 검증하는 테스트, 인증·인가 경계, OpenSearch 동적 쿼리를 직접 다뤘습니다. 이 경험은 이벤트 신뢰성, 배치 재시작, 피드 캐시와 검색 정책을 설계할 때의 기준이 되었습니다.

토스랩 · JANDI

2024.06 — 2025.04

커버리지 70%인데도 배포가 불안한 이유를 테스트 기준과 CI 실행 방식에서 찾았습니다. Drive-Service를 Spring Boot 2.3.8→3.3.3으로 올리며 테스트를 0→160개 이상으로 늘렸고, Mock 테스트가 놓친 JPA 변경 감지 오류는 실제 구현 테스트로 재현해 약 200만 건의 데이터를 보정했습니다.

오케스트로 · IDP / Vista

2023.09 — 2024.05

신용정보원 IDP에서 Spring Cloud API Gateway, Spring Security·JWT 인증/인가, SonarQube·Jenkins 연동 API를 개발했습니다. 하나금융그룹 Vista에서는 VM 메트릭 조회 API와 OpenSearch 동적 쿼리 분기를 구현했습니다.

즐거운 · Backend

2022.11 — 2023.02

자사 서비스의 백엔드 기능을 개발하고 운영 중 발생한 문제를 해결했습니다.

문제를 풀고 나면, 글과 코드로 다시 설명합니다.

업무에서 배운 내용을 혼자만의 요령으로 남기고 싶지 않았습니다. 테스트와 데이터 오류를 글로 풀고, 발표와 오픈소스 리뷰에서 팀 밖의 피드백도 받아 왔습니다.

YouthCon 2025 연사

‘우리팀을 위한 Spring Test 만들기’에서 테스트 코드를 줄이는 기준과 팀의 그라운드 룰을 만드는 과정을 발표했습니다.

Spring OSS · 개선 제안 3건 반영

Kafka, Boot, Data JPA에 제안한 개선 3건이 반영됐습니다. 불변 컬렉션 처리, 하드코딩 제거, 자원 정리 로직을 개선했습니다.

화면에 없던 7만+ 건의 오류

응답은 정상이었지만 S3에서는 삭제되고 DB에는 남은 데이터가 6년 동안 7만 건 넘게 쌓여 있었습니다. 이를 발견하고 원인을 좁혀 간 과정을 공개했습니다.

기술 글 156편 · 2026.07 기준

Java, JPA, 테스트, SQL 튜닝을 중심으로 시행착오를 기록했습니다. 글또 10기에서는 백엔드·인프라 빌리지 준비위원과 연사로 참여했습니다.

OpenSearch 한글 분석 플러그인

JavaCafe의 Elasticsearch 플러그인을 바탕으로 한·영 자판 변환, 초성·자모 검색을 OpenSearch 2.19와 Kotlin으로 옮기고 16개 예시로 확인했습니다.

테스트로 찾은 약 200만 건

Mock 테스트가 놓친 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다. 보정 스크립트를 작성해 약 200만 건의 데이터를 바로잡았습니다.